D5 - Signatures & Authentication

Start Assignment

- Due Sunday by 11:59pm
- Points 10
- Submitting a website url

Goal

Extend your system to guarantee **origin authentication and integrity** by implementing digital signatures.

After this deliverable, your system must allow:

- A recipient to verify *who created the file.*
- Detection of *any modification to the file or metadata.*
- Rejection of forged or tampered containers

Security Goal

A recipient must be able to verify that a file was created by a specific user and has not been modified since it was signed.

Up to D3, your system provides Confidentiality (encryption) and Access control (you decide who to share)

But it does NOT give a guarantee of who created the file or whether the sender is authentic.

Without signatures:

- Anyone can re-encrypt content and claim ownership
- Recipients cannot trust the origin

Basic Idea

You must implement:

Hash → Sign → Verify

What You Must Build

1. Digital Signature Generation

Your system must:

- Compute a *hash of the data to be signed*

- Sign that hash using the sender's *private key*

Recommended algorithms (we already review these in class):

- Ed25519 (preferred)
- ECDSA

2. Signature Verification

Before decrypting or processing a file, your system must:

- Verify the signature using the sender's public key
- Reject the file if verification fails

3. Define What Is Signed

You must explicitly define what data is included in the signature?

At minimum, you must include:

- Ciphertext
- All security-relevant metadata (same as AAD)

This prevents an attacker modifying metadata, an attacker changing recipient list and context manipulation attacks.

4. Integration with Existing System

Your signature must cover:

- Output from D2 (encrypted content)
- Structure from D3 (recipients, metadata)

Your system must follow this flow:

Encrypt → Add metadata → Sign → Store

5. Signature Storage

Your container must now include:

```
{
  metadata,
  recipients,
  ciphertext,
  tag,
  signature,
  signer_id
}
```

You have to define:

- How signer is identified (key ID, fingerprint, etc.).
- How signature is encoded (you already check the existing standards, use them).

6. Verification Workflow

Your system must follow these flow:

1. Read container
2. Extract metadata + ciphertext
3. Verify signature
4. IF valid then proceed to decrypt
5. IF invalid then reject

Required Tests

You have to create test to demonstrate:

A. Valid signature → file accepted
B. Modified ciphertext → rejected
C. Modified metadata → rejected
D. Wrong public key → rejected
E. Signature removed → rejected

Documentation Required

Signature Design

Explain:

- Algorithm chosen
- What data is signed
- Why hashing is required before signing

Security Decisions

Why sign ciphertext and not plaintext?
What happens if signature is not verified first?
What happens if metadata is excluded?